

Date: 8-6-26

Micro Service

* Three Types of micro Service

① Domain micro Service : (Business m)

- Customer Information, Service
- Manage Address
- Payment Gateway → upi, credit card, Refund
- Simple way to explain, its business ideas, how customer will buy & proceed

② Data micro Service

- product data, SKU
- Inventory (store info)

③ Infrastructure / Utility microservice

- Authentication

* Circuit Breaker

- If it is close then everything is fine, downstream working fine
- Open means downstream service is down.
- resilience kit is used dependency

Date : _____

Product layer
↓
Service layer (downstream)

Three - States in circuit breaker

→ Closed → Everything fine

→ Open → downstream Service is down

→ Half open → if ~~down~~ downstream is down, after some time later, Open will move to Half open & it checks, if success then close else open.

* Rate Limitor

→ if 10,000 request for a api, application can crash, if we set 100 request per second, it will reject the remaining calls.

* Kafka

→ Distributed messaging System

→ producer & consumer

→ Asynchronous Call - Don't wait, merge

Date: _____

based, Ex: kafka, RabbitMQ

* Synchronous Call

→ Rest Api → you wait for one service to complete & then serve the another class

* kafka vs Traditional messaging queue

→ kafka vs RabbitMQ, IBM MQ

* In a topic, once it consumed it still there

* Larger data

* multiple consumer can consume

in a queue, once it consumed it delete

* thousand of data

* only consumed once

* monolithic vs microservice

• Everything under one application

Product



Booking



Payment

• Separate microservice

→ product

→ Booking

→ payment

Date: _____

✗ Good for Small
team & Small
application
✗ Once application is
down, Everything is
down

✗ Good for larger
application

✗ One microservice is
down then other
microservice will run

How to handle long running Api

→ If the api taking long time it's
poor experience

→ Use Asynchronous method

→ If the payment gateway
application, Show message the payment
processing, In background proceed the
api work,

→ Use message broker, such as kafka

12-6-26

Out of memory

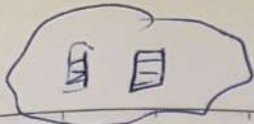
✗ Heap memory exhausted

✗ It's depleted heap

Stack overflow
VS Heap Memory

✗ stack memory
exhausted.

✗ it's recursive
thread.



Date: _____



Heap memory

vs

Stack memory

- Stores objects
- Shared along thread
- Manage by garbage collector
- Slower

- Stores local variable, method calls
- Each thread has its own stack
- Automatically cleared when method ends
- Faster.

Garbage Collection

→ It's a part of JVM
→ It removes the objects from the memory those are not used.

• How it identifies which is not used → Using Garbage Collector Root
if object A



→ Calls

Object B

if cannot reachable by GC Root
then mark it X, mark & sweep.

Date : _____

Dependency Injection

In a Spring boot application, I want to call a Employee class into company class. I need to create a object of Employee class and then I need to call a method. This is called Tightly Coupled.
with dependency Injection

→ we don't create a object of Employee class, we call the Employee class based on constructor. This is loosely coupled. Here we no need to change the business logic in future if required. Easier to modify.

Method overloading

→ add(int a, int b)

→ add(int a, int b, int c)

method name can be same, but parameter will be different.